Blocked-Credit Guard: AI-assisted GST input-tax-credit checks for Indian SMEs

Jurisdiction: India (CGST Act 2017, Sections 16 and 17(5), as amended by Finance Act 2025). **Data:** the rules are taken from public statute text, and the 14 invoice lines are synthetic (see `data/SOURCES.md`). **Prototype:** live demo (linked in the submission form) and a GitHub repository.

1. Area explored and what surprised me

I explored **GST input tax credit (ITC)**. A business deducts the GST it paid on purchases from the GST it owes on sales. This is allowed only if the conditions in Section 16 hold and the purchase is not on the Section 17(5) "blocked" list.

What surprised me:

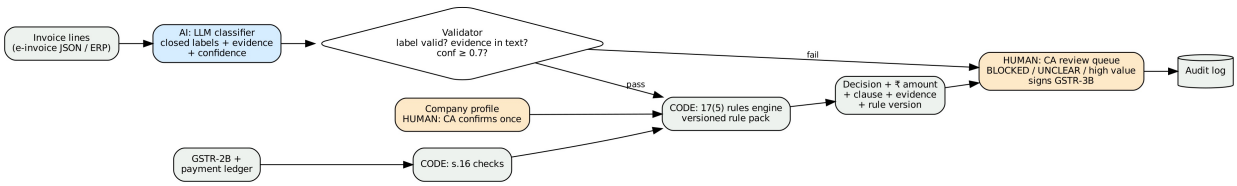
- **The law changes backwards in time.** In *Safari Retreats* the Supreme Court read "plant **or** machinery" in s.17(5)(d) broadly, which allowed credit on some buildings. The Finance Act 2025 then changed the wording to "plant **and** machinery" with effect from 1 July 2017, reversing that ruling for every year since GST began. A decision that was correct last year can be wrong today for the same invoice.
- **Eligibility depends on who the buyer is, not only on what was bought.** The same Innova car is blocked for a plastics manufacturer and eligible for a cab operator. A lookup table by item or HSN code cannot capture this.
- **Invoices are written in the seller's words, not the statute's.** "Buffet for 300 pax" is food and "Corporate wellness plan" is a gym membership, but neither text contains the word the law uses.

2. Problem statement

- **Who:** accountants and junior CA staff at small and mid-size GST-registered businesses, typically handling a few hundred purchase-invoice lines a month.
- **Task:** before filing GSTR-3B each month, decide line by line whether the ITC can be claimed. GSTR-2B matching is already automated by many tools. The **17(5) blocked-credit judgement** is still made by a person reading line descriptions.
- **How often:** every month, plus the annual GSTR-9 reconciliation, where 17(5) reversals are reported in Table 7E.
- **What it costs today:**
  - A wrongly claimed credit must be reversed with interest, and penalties can follow at scrutiny or audit.
  - A missed eligible credit is lost for good after the s.16(4) deadline (30 November after the financial year ends).
- **Why now:**
  - Since 2022, s.16(2)(aa) makes GSTR-2B the legal basis for ITC, so matching is a solved problem and judgement is what remains.
  - The 2025 retroactive amendment shows the rules keep moving.
  - Low-cost LLMs can now read messy text reliably enough to help.

Out of scope: GSTR-2B matching, Rule 42/43 apportionment for mixed use, reverse charge, filing returns, OCR of invoice PDFs (the input is structured line text from an ERP or e-invoice JSON), and any client-facing advice.

3. Solution design



Architecture: blue = AI, grey = deterministic code, orange = human

Data in:

- the invoice line text and its tax amount (the `ItemList.PrdDesc` field of the e-invoice JSON);
- GSTR-2B status and payment dates;
- a one-time **company profile**, confirmed by a CA: business activity, whether the business supplies the same category onward, and any employee benefits that are mandated by law.

What the AI does, and nothing more: it maps a line to one of 14 fixed categories, returns an evidence string that must appear word for word in the source, and gives a confidence score. It never decides eligibility and never computes tax.

Deterministic and auditable: the s.16 checks, the 17(5) blocks and their exceptions, and the rupee amount. All of this is plain Python that a CA can read line by line.

Output for each line:

- a status: `ELIGIBLE`, `BLOCKED`, `ELIGIBLE_EXCEPTION`, `NOT_ELIGIBLE_S16` or `HUMAN_REVIEW`;
- the eligible amount, the clause, the evidence quote and the rule-pack version.

Every output can be traced back to a statute clause and to the exact words on the invoice.

Human in the loop: the CA, or the authorised signatory who signs GSTR-3B, approves the monthly batch. Every `BLOCKED` and `HUMAN_REVIEW` line, and every line above a value threshold, is reviewed individually. The tool prepares the work; the professional decides and remains accountable.

How uncertainty and mistakes are handled:

- A label outside the fixed set, evidence that does not appear in the source, or confidence below 0.7 sends the line to human review. The system never guesses in those cases.
- The default is conservative because the two errors cost different amounts. A false "eligible" means reversal plus interest plus a penalty. A false "blocked" can still be corrected until the s.16(4) deadline.

Where the knowledge lives:

- **Law:** in a versioned rule pack in code, with the version stamped on every decision. When the law changes, the pack is updated and past invoices are re-run to produce a list of claims that need revisiting. That is how the 2025 retroactive change would be handled.
- **The model:** general language understanding only, no tax law.

- **Facts about the business:** in the company profile.

Why AI is needed: a non-AI keyword baseline scored only **7/14**, and its errors went both ways.

- It wrongly blocked car brake pads bought for resale and a machine foundation.
- It missed food, gifts, a gym membership and CSR spending when they were described in everyday words.

Adding more keywords trades one error for the other. Only a model that understands meaning generalises across how different suppliers write, and limiting it to labelling keeps its risk contained.

4. Prototype: how it works

The prototype has four parts:

- **Live demo (`demo.html`, published link):**
  - Pick any of the 14 synthetic cases, or type your own line.
  - Set the buyer's business activity and the GSTR-2B and payment flags, then press *Check ITC*.
  - The page shows the decision, the rupee amount, the clause, the evidence and a 5-step audit trail (classifier → s.16 → 17(5) → decision → CA queue).
  - The "Classifier label" menu lets you see what the rules engine does with a correct LLM label. A full table runs the baseline on all 14 cases.
- **`run_tests.py`:** runs every case with the keyword baseline, or with `--llm` (an OpenRouter free model, temperature 0, JSON output, validated), and writes `results_*.json`.
- **`app.py`:** a Streamlit interface for the same pipeline that calls the real LLM when an API key is set.
- **`rules.py`:** the rule pack, roughly 100 readable lines.

Test set (synthetic)	Keyword baseline	Where it failed
14 lines: plain inputs, cars (two buyer types), food, gifts, gym, CSR, civil work, a machine foundation, s.16 failures, vague text	**7/14**	#1 sent to review needlessly; #4, #5, #12, #14 missed (no keyword); #6 and #11 falsely blocked

Where it still fails, honestly:

- **#6, a foundation for a machine.** Whether this counts as plant and machinery is genuinely contested in law. The correct behaviour is to send it to a human, not to give a confident answer.
- **#10, "Professional charges as per agreement".** This must come back as `unclear`. If a model guesses here, that counts as a failure.
- **Test set size.** 14 cases are too few to estimate accuracy for each category.

5. Alternatives considered and rejected

Option	Why rejected
A single LLM call that decides "eligible or not"	Not auditable, mixes law with guessing, amounts can be hallucinated, and there is no way to roll a rule change forward
RAG over the CGST Act at runtime	17(5) is short and stable enough to encode. Retrieval adds a failure mode (the wrong clause) without adding coverage. Worth revisiting for circulars and advance rulings.
Lookup by HSN/SAC code	The same code can be blocked or eligible depending on the buyer, and codes are often missing or wrong on SME invoices
Multi-agent design	Nothing here needs negotiation between agents. It adds cost and is hard to explain to an auditor.
Letting the LLM infer the exceptions	Exceptions depend on facts about the business, not on the line text, so a human confirms them once

6. Risks and how the design handles each

- **Wrong answers.** Controlled by the fixed label set, the evidence check, the confidence threshold, the value threshold and CA sign-off. Errors should be measured per category before go-live.
- **Outdated rules.** The rule pack is versioned with effective dates and past invoices are re-run when it changes. The weak point is noticing an amendment in the first place, which could be addressed with a feed of regulatory updates later.
- **Confidential data.** Only the line text and the business activity go to the model, never GSTINs or party names. Production needs a self-hosted model or one with zero data retention.
- **Liability.** The output is labelled as an aid to preparation. The registered person and their CA stay responsible, and the audit log records who approved each line.
- **Weakest point.** A wrong company profile would cause the same silent error every month. Mitigations: re-confirm the profile every year, and raise an alert when the share of blocked lines changes sharply.

7. Before spending more time

- Validate the problem with two or three practising CAs: are 17(5) errors what actually trigger notices, or is it suppliers not filing their returns?
- Build a larger, realistic set of line descriptions.
- Check whether Tally, Zoho or ClearTax already offer this, and position the tool as a plug-in.
- Measure the review rate. If more than about 30% of lines go to a human, the tool saves too little time to be worth it.

Appendix: What I added beyond the AI

Where AI helped: speed work — surveying GST/TDS/RPT problem spaces, verifying the latest statute position (Finance Act 2025 change to s.17(5)(d); s.16(2)(aa), 16(4)), generating boilerplate code and the HTML demo, and typesetting this brief. Every design decision below was set by me and the AI implemented it.

Key prompts (from the attached chat log):

1. *Scoping:* "Use the assignment PDF; shortlist India-focused finance problems that avoid the two given examples, with who/task/cost and a proposed architecture for each." I then picked GST ITC over TDS, RPT audit and tax-notice explainers.
2. *Grounding:* "First get real data. Every rule must trace to public statute text, and the data must be a separate file or link referenced in the doc; label anything synthetic." This produced `data/SOURCES.md` and a labelled synthetic CSV.
3. *Proof:* "Build an actual working prototype, not a write-up, and explain in the brief exactly how it runs." This produced the clickable demo with an audit trail, the Streamlit app and the test runner.

What I rejected or corrected:

- **RAG over the CGST Act.** The AI's default for "finance documents". I rejected it: 17(5) is about a dozen fixed clauses, so retrieval only adds a wrong-clause failure mode. Encoding it as a versioned rule pack is smaller, exact and auditable.
- **Letting the LLM output "eligible/not eligible".** Rejected as unauditable. A hallucinated rupee figure on a GSTR-3B is a legal exposure. I restricted the model to labelling.
- **HSN-code lookup as the non-AI option.** Rejected after the Innova case: the same code is blocked or allowed depending on the buyer.
- **Generic test cases.** I asked for adversarial ones instead: food without the word "food", a "car" keyword on resale parts, and a legally contested machine foundation.

What came from me:

1. **Exact vs probabilistic split.** I chose ITC because it separates cleanly: reading text is probabilistic and goes to the LLM; the clause, amount and deadline are exact and go to code. The two never mix.
2. **Buyer context is a confirmed input, not an inference.** Eligibility depends on who buys (#2 vs #3, same car). A CA confirms the company profile once, which removes a whole class of silent model errors.
3. **Asymmetric error costs set the threshold.** A false "eligible" means reversal + interest + penalty, while a false "blocked" is recoverable until 30 November. So uncertainty defaults to review, and I set confidence at 0.7.
4. **Designing for retroactive law.** Finance Act 2025 rewrote a 2017 rule after *Safari Retreats*. I made the rule pack versioned and stamped on every decision, so past periods can be re-run and diffed. Most designs only handle forward changes.
5. **A baseline as a control experiment.** Instead of asserting that AI is needed, I measured it: the keyword rules scored 7/14 with errors in both directions, which more keywords cannot fix.
6. **Evidence-span guardrail.** The model must quote text that actually exists on the invoice, or the line goes to a human. This makes every label checkable by an auditor in seconds.

One thing I'd defend without AI: why the LLM only labels and code decides. Live, I can walk through #2 vs #3 (same car, different buyer → 17(5)(a) and its exception), #6 (machine foundation: plant and machinery vs immovable property after the 2025 amendment → routed to a human), and the s.16(4) deadline arithmetic.